VulnLens

Smart Contract Security Analysis Report

Contract Information

Contract	reentrancy.sol
Solidity Version	^0.8.20
Source Lines	24
Generated	17 October 2026, 03:22

Executive Summary

VulnLens identified **3** finding(s) in the submitted Solidity contract. Findings are ranked from High to Medium to Low to Informational. Each finding contains static evidence, relevant source context, contextual reasoning, and remediation guidance.

High	Medium	Low	Informational
1	**0**	**0**	**2**

Ranked Security Findings

Finding 1 — High

Detector	reentrancy-eth
Affected Function	withdraw
Detected Source Lines	31, 39
Static Confidence	Medium

Static Analysis Evidence

• (success,None) = msg.sender.call{value: amount}() (reentrancy.sol#31)

• balances[msg.sender] = 0 (reentrancy.sol#39)

• ReentrancyVulnerable.balances (reentrancy.sol#9)

• ReentrancyVulnerable.deposit() (reentrancy.sol#13-17)

• ReentrancyVulnerable.withdraw() (reentrancy.sol#21-41)

Relevant Solidity Context

```
1: // SPDX-License-Identifier: MIT
2: pragma solidity ^0.8.20;
3:
4: contract ReentrancyVulnerable {
5:     mapping(address => uint256) public balances;
6:
7:     function deposit() external payable {
8:         balances[msg.sender] += msg.value;
9:     }
10:
11:     function withdraw() external {
12:         uint256 amount = balances[msg.sender];
13:
14:         require(amount > 0, "No balance");
15:
16:         (bool success, ) = msg.sender.call{value: amount}("");
17:
18:         require(success, "Transfer failed");
19:
20:         balances[msg.sender] = 0;
21:     }
22:
23:     receive() external payable {}
24: }
```

Contextual LLM Assessment

The withdraw function sends ETH via msg.sender.call before setting the user's balance to zero, which follows the checks■effects■interactions anti■pattern and enables a reentrancy attack.

Exploitability: Qualified: an attacker can deploy a malicious contract that calls withdraw and, in its receive/fallback function, re■enters withdraw repeatedly before the balance is cleared, allowing the attacker to drain the contract's ETH.

Attack Scenario: Attacker deposits some ETH into the contract, then calls withdraw from a malicious contract. The malicious contract's receive function triggers another call to withdraw, which repeats until the contract's ETH balance is exhausted.

Remediation Steps

1. Update the user's balance before making the external call (checks■effects■interactions pattern).
2. Use a non■reentrant modifier (e.g., OpenZeppelin's ReentrancyGuard) to protect the withdraw function.
3. Consider using a pull■over■push pattern where users withdraw funds via a separate function after initiating a withdrawal request.

Finding 2 — Informational

Detector	solc-version
Affected Function	Not available
Detected Source Lines	3
Static Confidence	High

Static Analysis Evidence

• VerbatimInvalidDeduplication

• FullInlinerNonExpressionSplitArgumentEvaluationOrder

• MissingSideEffectsOnSelectorAccess.

• ^0.8.20 (reentrancy.sol#3)

Relevant Solidity Context

```
1: // SPDX-License-Identifier: MIT
2: pragma solidity ^0.8.20;
3:
4: contract ReentrancyVulnerable {
5:     mapping(address => uint256) public balances;
6:
7:     function deposit() external payable {
8:         balances[msg.sender] += msg.value;
```

Contextual LLM Assessment

The contract uses Solidity version constraint ^0.8.20, which is flagged by the solc-version detector as containing known severe compiler bugs. The reported bugs include VerbatimInvalidDeduplication, FullInlinerNonExpressionSplitArgumentEvaluationOrder, and MissingSideEffectsOnSelectorAccess. These are documented compiler-level issues that could lead to incorrect code generation or unexpected behavior during compilation or execution. The pragma statement is located at reentrancy.sol line 3. While the provided code snippet only shows a simple deposit function, the use of a vulnerable compiler version affects the entire contract and any compiled bytecode derived from it.

Exploitability: Exploitability depends on whether the contract's logic triggers any of the affected compiler bugs. The bugs listed are related to internal compiler optimizations and code generation, meaning they could manifest under specific conditions involving complex expressions, function inlining, or selector access patterns. Without full contract code, it cannot be definitively confirmed that these bugs are exploitable in this specific contract. However, the risk exists due to the use of a known vulnerable compiler version.

Attack Scenario: An attacker could potentially exploit one of the compiler bugs if the contract's logic involves patterns that trigger the affected optimizations. For example, if the contract uses complex expressions or selector-based calls that interact with the FullInlinerNonExpressionSplitArgumentEvaluationOrder or MissingSideEffectsOnSelectorAccess bugs, the resulting bytecode might behave differently than intended. This could lead to unintended state changes, incorrect balance updates, or bypassed access controls. However, based solely on the provided deposit function, there is insufficient evidence to confirm an active exploit path.

Remediation Steps

1. Upgrade the Solidity compiler version to a patched release that does not contain the known severe bugs. For example, use a version later than 0.8.21 if available, or consult the Solidity release notes to identify a safe version.
2. Review the full contract code to ensure no logic patterns trigger the affected compiler bugs, particularly around expression evaluation order, function inlining, and selector access.
3. Recompile the contract with the updated compiler version and perform thorough testing, including unit tests and formal verification, to confirm correct behavior.

Finding 3 — Informational

Detector	low-level-calls
Affected Function	withdraw
Detected Source Lines	31
Static Confidence	High

Static Analysis Evidence

• (success,None) = msg.sender.call{value: amount}() (reentrancy.sol#31)

Relevant Solidity Context

```
1: // SPDX-License-Identifier: MIT
2: pragma solidity ^0.8.20;
3:
4: contract ReentrancyVulnerable {
5:     mapping(address => uint256) public balances;
6:
7:     function deposit() external payable {
8:         balances[msg.sender] += msg.value;
9:     }
10:
11:     function withdraw() external {
12:         uint256 amount = balances[msg.sender];
13:
14:         require(amount > 0, "No balance");
15:
16:         (bool success, ) = msg.sender.call{value: amount}("");
17:
18:         require(success, "Transfer failed");
19:
20:         balances[msg.sender] = 0;
21:     }
22:
23:     receive() external payable {}
24: }
```

Contextual LLM Assessment

The withdraw function uses a low-level call (msg.sender.call{value: amount}) before setting the user's balance to zero, and the contract defines a receive() fallback that can be triggered by the called contract, enabling a reentrancy attack.

Exploitability: High – an attacker can deploy a malicious contract that calls withdraw and implements receive() to re-enter withdraw, allowing multiple withdrawals before balance is cleared.

Attack Scenario: 1. Attacker deposits Ether into ReentrancyVulnerable. 2. Attacker calls withdraw() from their malicious contract. 3. The low-level call triggers the malicious contract's receive() fallback. 4. In receive(), the attacker calls withdraw() again. 5. Since balances[msg.sender] hasn't been set to zero yet, the second call withdraws again, repeating until funds are drained.

Remediation Steps

1. Move the balance update (balances[msg.sender] = 0;) before the external call (checks-effects-interactions).

2. Alternatively, use OpenZeppelin's nonReentrant modifier to protect the withdraw function.

Analysis Methodology

• The Solidity source file is validated before analysis.

• Slither performs static analysis on the submitted contract.

• Detector results are converted into structured security evidence.

• Relevant Solidity source context is selected around reported locations.

• High, medium, and low impact findings are supplied to the configured LLM for contextual reasoning.

• Findings are ranked from High to Medium to Low to Informational.

• The final report combines static-analysis evidence, source context, contextual reasoning, and remediation guidance.

Scope and Limitations

• VulnLens is a pre-deployment smart-contract analysis system.

• The system does not deploy contracts.

• The system does not interact with a blockchain or ledger.

• The system does not use wallets or private keys.

• Static-analysis findings identify suspicious or security-relevant patterns and require contextual review.

• LLM output is contextual reasoning over supplied evidence and source code and should not replace independent security review.

• The absence of a finding does not prove that a contract is completely secure.

Generated by VulnLens